Embeddings: Turning Text into Meaningful Numbers

A short learning document for experimenting with a Retrieval-Augmented Generation (RAG) pipeline.

1. What is an embedding?

An embedding is a list of numbers (a vector) produced by a machine-learning model from data such as a sentence, paragraph, image, or audio clip. For text, the vector is designed to represent useful semantic features of the text. Texts with related meanings often have vectors that are close to each other in the embedding space.

2. Why does RAG use embeddings?

A RAG system first extracts text from documents and splits it into smaller pieces called chunks. An embedding model converts each chunk into a vector. The vectors and their associated text are stored in a vector database such as ChromaDB. When a user asks a question, the system embeds the question and searches for chunks with similar vectors. The retrieved text is then supplied to a language model as context.

3. A simple example

Imagine a knowledge base containing these sentences:

- **Chunk A:** “A cat is a small domesticated animal.”
- **Chunk B:** “A kitten is a young cat.”
- **Chunk C:** “A database stores and organizes information.”

For the question “What is a young cat called?”, a semantic retriever may rank Chunk B highly, even though the question does not use exactly the same words. This is one benefit of embeddings: retrieval can use meaning, not only exact keyword matches.

4. Similarity and vector search

A vector database compares the query vector with stored vectors using a distance or similarity measure. Cosine similarity is one common measure. The exact score and ranking depend on the embedding model, vector normalization, and database configuration. A high similarity score is a useful signal, but it does not guarantee that a chunk answers the question correctly.

5. Important terms

- **Embedding model:** the model that converts data into vectors.
- **Chunk:** a small section of a source document.
- **Vector store:** a system that stores vectors and supports similarity search.
- **Retriever:** the component that selects relevant chunks for a query.
- **Metadata:** additional information such as source filename, page number, or chunk index.

6. How to test this in a RAG project

- Load this PDF with your PDF loader and inspect the extracted text.
- Split the text into chunks and create an embedding for each chunk.

- Store the chunks, embeddings, and metadata in ChromaDB.
- Ask: “How does RAG find relevant document passages?”
- Check whether the retrieved chunks mention query embeddings, vector similarity, and retrieval.

Remember: embeddings help find relevant information. They do not generate the final natural-language answer by themselves.